

1.What is Inter Process Communication[IPC]:

Inter process communication is how separate processes on the same machine share data and coordinate with each other. Each process normally runs in its own protected memory space, so one can't just read another's variables. The operating system has to provide a channel between them, and IPC is the name for those channels. You need it whenever a job is split across processes. A web server might hand requests to worker processes, a browser keeps each tab in its own process, and a shell passes one command's output into the next with a pipe.

2.IPC Techniques:

There are in total eight IPC techniques in which 2 are as follows :

(i)Shared Memory:

Definition: Shared memory is a block of RAM that two or more processes can both read and write. Normally each process gets its own private address space, so the OS blocks one from touching another's memory. With shared memory, the OS deliberately sets aside one region and maps it into each process's address space.

Working: One process asks the OS to create a shared segment. The OS hands back an ID (or a name). Other processes use that ID to attach the segment, and each gets a pointer to it. After that, a write by one process is immediately visible to the others, because they're all looking at the same physical memory. This is why it's the fastest form of IPC. Pipes and message queues copy data into the kernel and back out again. Shared memory skips that copying, and the kernel only gets involved during setup and cleanup. The catch is that the OS won't stop two processes from writing at the same time, so you have to add your own synchronization, usually a semaphore or mutex. Without it you get race conditions.

The classic System V calls go in this order:

- shmget()- creates or finds the segment
- shmat()- attaches it to your process
- shmdt()- detaches it
- shmctl()- removes it when you're done

Example: Video player-One process decodes the video frames and drops them into a shared buffer, and another process reads from that buffer and draws them on screen. Copying every frame from one process to the other would be slow, so they share the memory instead; Browsers do something similar, sharing memory between the main process and the tab processes for things like images and rendering data.

(ii) Message Queues:

Definition: A message queue is a list of messages that the operating system holds on behalf of processes. One process puts a message in, and another takes it out later. The kernel stores the messages in between, so the sender and receiver don't have to be running at the same moment. Think of a mailbox. You drop a letter in and leave. The other person collects it whenever they get around to it. Unlike a pipe, which is a raw stream of bytes, a message queue keeps each message as a separate unit, and each one can carry a type number.

Working: One process creates a queue, and the OS returns an ID for it. Any process that knows the key can get the same ID and use the queue. A sender adds a message, and the kernel copies it into the queue and returns right away. A receiver asks for a message, and the kernel copies it out and removes it from the queue. If the queue is empty, the receiver normally blocks until something arrives. Messages come out in first-in, first-out order by default, but a receiver can ask for a specific type and skip past the others. That's handy when one queue serves several processes.

The System V calls are:

- `msgget()`- creates or opens a queue
- `msgsnd()`- sends a message
- `msgrcv()`- receives one
- `msgctl()`- removes the queue when you're done

Compared with shared memory, message queues are slower because every message gets copied into the kernel and back out. In return you get built-in safety. The kernel handles the ordering and locking, so you don't need your own semaphores to keep two processes from colliding.

Example: Food delivery app- When you place an order, the app doesn't stand there waiting for the restaurant. It puts the order in a queue, and the restaurant's system picks it up when the kitchen is ready. If the restaurant's system is briefly down, the order just sits there until it comes back.